



Density-Based Models

The Advanced R Series: Clustering & Classification

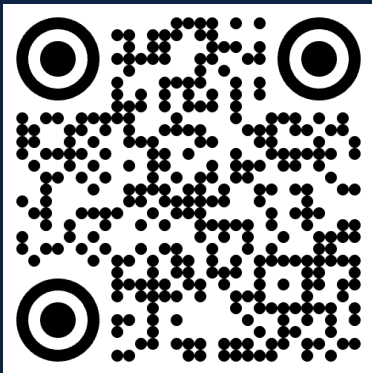
Slides and code created by Jesse Ghashti.

Presented by Alex.

February 9, 2026

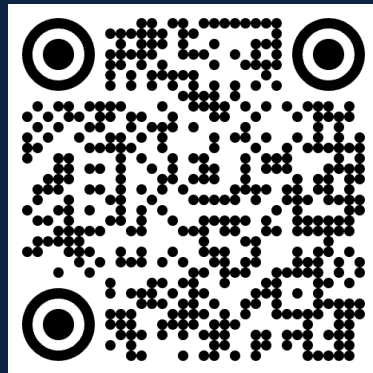
Centre for Scholarly Communication

The University of British Columbia | Okanagan Campus | Syilx Okanagan Nation Territory



←
Check out our other
CSC workshops!

GitHub code and
slides for today's
workshop (and pre-
vious workshops)



Alternatively, code/slides available at the bottom of
<https://csc-ubc-okanagan.github.io/workshops/>

Workshop Series Overview



Session	Topic	Date/Time
1	Hierarchical Models	Jan 14, 12:00 PM
2	Centroid-Based Models	Jan 19, 3:30 PM
3	Fuzzy Clustering	Jan 28, 1:00 PM
4	Distribution-Based Models	Feb 2, 3:30 PM
5	Density-Based Models	Feb 9, 3:30 PM
6	Graph-Based Models	Feb 23, 3:30 PM
7	Mixed-Type Data Quantification	Mar 4, 1:00 PM
8	Mixed-Type Data Clustering	Mar 11, 1:00 PM
9	Bias Reduction and Fairness	Mar 18, 1:00 PM
10	Dimensionality Reduction 1 of 2	Mar 23, 3:30 PM
11	Dimensionality Reduction 2 of 2	Mar 30, 3:30 PM

What we learned...

- Learned Gaussian Mixture Models (GMMs) and the EM algorithm
- Explored covariance structures and 95% confidence ellipses
- Used BIC and ICL for model and component selection
- Introduced Dirichlet Process Mixture Models (DPMM)
- Applied GMM to breast cancer diagnosis with risk stratification
- Compared posterior probabilities to fuzzy memberships from Workshop 3

Today -> Density-Based Clustering



Today we will...

- Understand why distributional assumptions still limit clustering
- Learn DBSCAN: core points, border points, and noise
- Select parameters using the k -distance plot
- Introduce OPTICS and HDBSCAN as alternative methods.
- Apply density-based methods to Chicago crime hotspot detection

Today we require...

```
library('dbscan')           # DBSCAN, OPTICS, HDBSCAN
library('factoextra')
library('ggplot2')
library('dplyr')
library('gridExtra')
library('viridis')
library('fpc')              # clustering validation
library('ggforce')
library('data.table')
```

Limitations We Have Addressed

- **Workshop 2** (K -means): Hard cluster boundaries + spherical assumption
- **Workshop 3** (FCM): Addressed hard boundaries with soft memberships
- **Workshop 4** (GMM): Relaxed spherical assumption with per-cluster covariance

But GMM still assumes

- Data comes from a mixture of Gaussian distributions
- Clusters are elliptical in shape
- Number of clusters must be specified or selected via BIC
- No explicit handling of outliers/noise

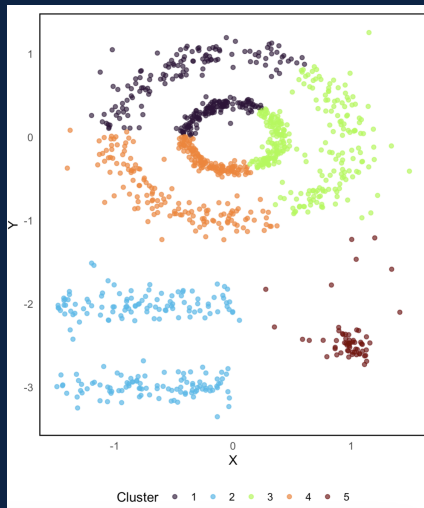
What if we drop distributional assumptions entirely and focus on **density**?

K -means on Arbitrary Shapes



```
data("multishapes")
shapesDf <- data.frame(
  x = multishapes[, 1],
  y = multishapes[, 2],
  trueCluster = factor(multishapes[, 3]))
kmeansShapes <- kmeans(
  shapesDf[, c("x", "y")],
  centers = 5, nstart = 25)
shapesDf$kmeansCluster <-
  factor(kmeansShapes$cluster)
```

You're likely sick of hearing this by now, but K -means assumes spherical clusters. It cannot recover rings, arcs, or irregularly shaped clusters.

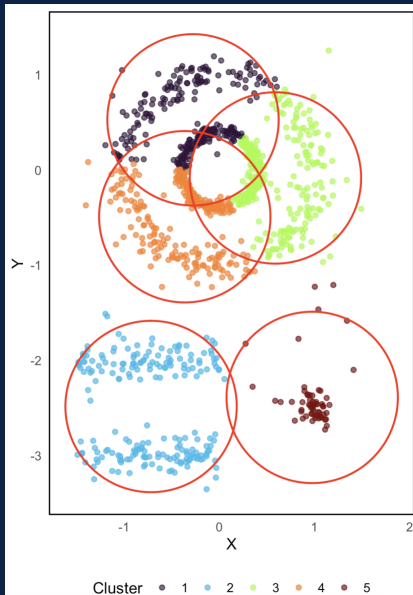


K-means Spherical Boundaries



```
centers <- as.data.frame(kmeansShapes$centers)
names(centers) <- c("x0", "y0")
centers$r <- 0.9
```

```
ggplot(shapesDf, aes(x = x, y = y,
                     color = kmeansCluster)) +
  geom_point(size = 1.5, alpha = 0.7) +
  geom_circle(
    data = centers,
    aes(x0 = x0, y0 = y0, r = r),
    inherit.aes = FALSE,
    color = "red", linewidth = 0.7) +
  coord_fixed() +
  scale_color_viridis_d(option = "turbo")
```



Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

Idea is that clusters are dense regions separated by sparse regions.

Two parameters

- ϵ (eps): radius of neighbourhood around a point
- minPts: minimum number of points to form a dense region

Point classification

- **Core point:** has $\geq \text{minPts}$ neighbours within ϵ radius
- **Border point:** within ϵ of a core point, but $< \text{minPts}$ neighbours
- **Noise point:** neither of above $\rightarrow$ classified as outlier

A huge bonus is that there is no need to specify the number of clusters. Outliers are detected automatically.

Steps

For each unvisited point p

1. Find all neighbours within ε radius
2. If $|\text{neighbours}| \geq \text{minPts}$: p is a core point. With this core point
 - Create a new cluster
 - Add p and all density-reachable points to the cluster
3. Otherwise mark p as noise (may later become a border point) and move onto the next unvisited point

Density-reachable: point q is density-reachable from p if there exists a chain of core points from p to q , each within ε of the next.

Core Points and Neighbourhoods

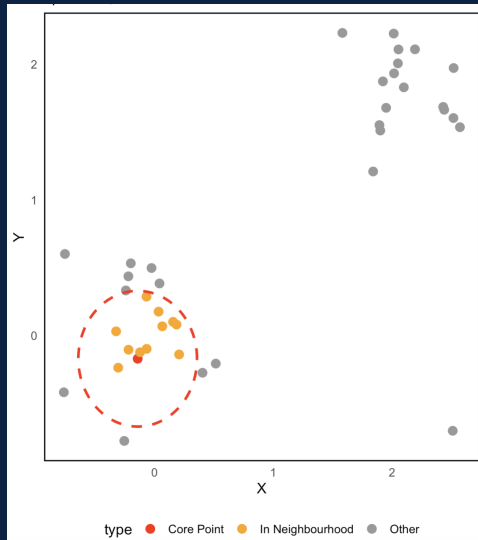


```
set.seed(2026)
conceptData <- data.frame(
  x = c(rnorm(20, 0, 0.3), rnorm(15, 2, 0.3),
        runif(5, -1, 3)),
  y = c(rnorm(20, 0, 0.3), rnorm(15, 2, 0.3),
        runif(5, -1, 3))
)
corePointIdx <- 10
eps <- 0.5
minPts <- 4
distances <- sqrt(
  (conceptData$x - conceptData$x[corePointIdx])^2 +
  (conceptData$y - conceptData$y[corePointIdx])^2
)
inNeighbourhood <- distances <= eps

conceptData$type <- "Other"
conceptData$type[inNeighbourhood] <- "In Neighbourhood"
conceptData$type[corePointIdx] <- "Core Point"

nNeighbours <- sum(inNeighbourhood) - 1
```

The red point has $\geq \text{minPts}$ neighbours within ε , so it is a core point. Orange points are in its neighbourhood.



```
shapesMatrix <- as.matrix(shapesDf[, c("x", "y")])
dbscanResult <- dbSCAN(shapesMatrix,
                        eps = 0.15,
                        minPts = 5)

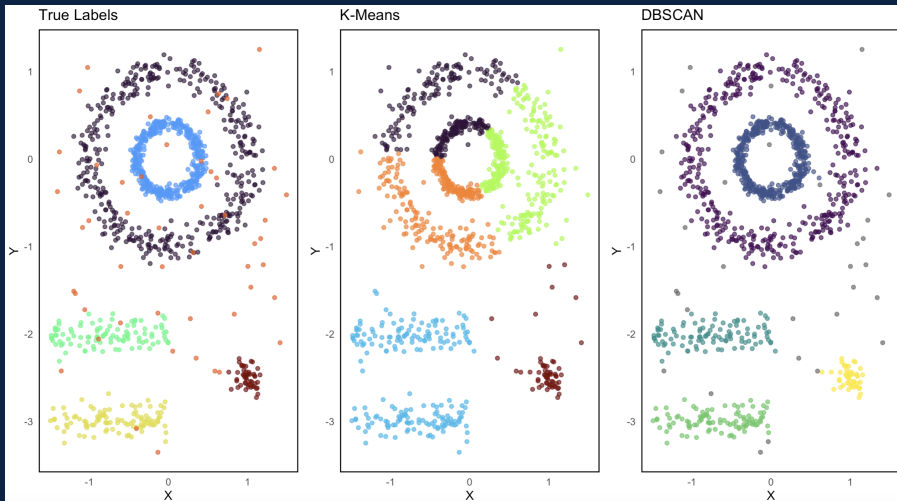
max(dbscanResult$cluster)
sum(dbscanResult$cluster == 0)
print(table(dbscanResult$cluster))

shapesDf$dbscanCluster <-
  factor(dbscanResult$cluster)
```

- 5 clusters found (no K specified)
- 31 noise points detected
- Cluster 0 = noise/outliers
- Looks pretty good!

	ARI	NMI
K -Means	0.216	0.436
DBSCAN	0.963	0.934

True Labels, K -means and DBSCAN

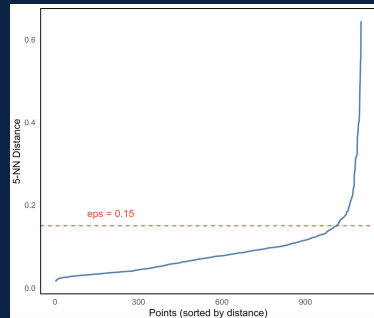


We select ε using the k -distance plot. The procedure works as follows:

1. For each point, compute the distance to its k -th nearest neighbour
2. Sort these distances in ascending order
3. Plot and look for the **elbow** (similar to WCSS for K -means)
4. The elbow suggests a good ε value

The rule of thumb is to set $k = \min Pts$

```
k <- 5
knnDist <- kNNdist(shapesMatrix, k = k)
sortedDist <- sort(knnDist)
kDistDf <- data.frame(index = 1:length(sortedDist), distance = sortedDist)
ggplot(kDistDf, aes(x = index, y = distance)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  geom_hline(yintercept = 0.15,
             linetype = "dashed", color = "red")
```



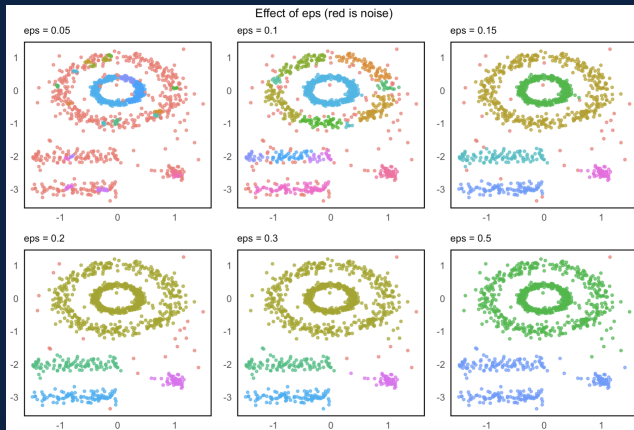
Effect of ϵ



```
epsValues <- c(0.05, 0.10, 0.15,
              0.20, 0.30, 0.50)
epsResults <- list()

for (epsVal in epsValues) {
  result <- dbSCAN(shapesMatrix,
                  eps = epsVal,
                  minPts = 5)
  epsResults[[as.character(epsVal)]] <- list(
    eps = epsVal,
    nClusters = max(result$cluster),
    nNoise = sum(result$cluster == 0),
    clusters = result$cluster)
}
```

eps	# Clusters found	# Noise points
0.05	21	569
0.10	15	118
0.15	5	31
0.20	4	21
0.30	4	7
0.50	2	1



Addressing DBSCAN's Sensitivity to ε

We have a (potential) problem... DBSCAN requires a single global ε , but clusters may have different densities.

OPTICS can help with this limitation:

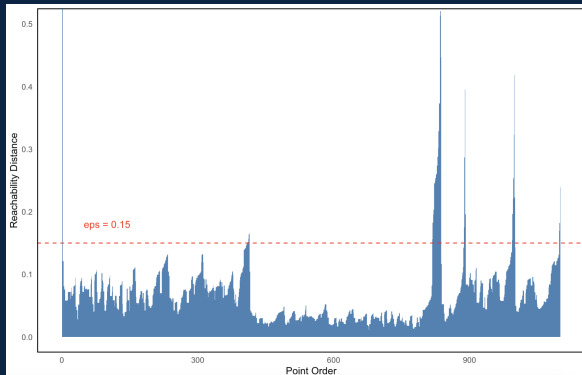
- Creates an ordering of points based on reachability distance
- Does not produce clusters directly; produces a **reachability plot**
- Clusters can be extracted at *any* ε cut from the ordering

Valleys in the reachability plot correspond to clusters; peaks correspond to cluster boundaries.

OPTICS Reachability Plot



```
opticsResult <- optics(shapesMatrix,  
                      eps = 1, minPts = 5)  
  
reachDf <- data.frame(  
  order = 1:length(opticsResult$order),  
  reachability = opticsResult$reachdist[  
    opticsResult$order])  
  
maxReach <- max(reachDf$reachability[  
  is.finite(reachDf$reachability)])  
reachDf$reachability[  
  !is.finite(reachDf$reachability)] <-  
  maxReach * 1.1  
  
ggplot(reachDf, aes(x = order,  
                    y = reachability)) +  
  geom_bar(stat = "identity", width = 1,  
          fill = "steelblue") +  
  geom_hline(yintercept = 0.15,  
            linetype = "dashed", color = "red") +  
  coord_cartesian(ylim = c(0, 0.5))
```



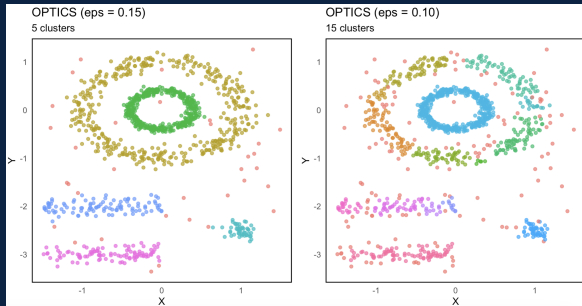
Valleys = clusters. Peaks = boundaries. Cutting at different heights gives different clusters.

Extracting Clusters from OPTICS



```
opticsDbscan015 <- extractDBSCAN(  
  opticsResult, eps_cl = 0.15)  
opticsDbscan010 <- extractDBSCAN(  
  opticsResult, eps_cl = 0.10)  
  
shapesDf$opticsCluster015 <-  
  factor(opticsDbscan015$cluster)  
shapesDf$opticsCluster010 <-  
  factor(opticsDbscan010$cluster)
```

extractDBSCAN cuts the reachability plot at a given ε , producing DBSCAN-like clusters.
Different cuts $\rightarrow$ different resolutions of the same ordering.



HDBSCAN combines

- DBSCAN's benefit of finding arbitrary-shaped clusters
- Hierarchical clustering's multi-scale view (Workshop 1)

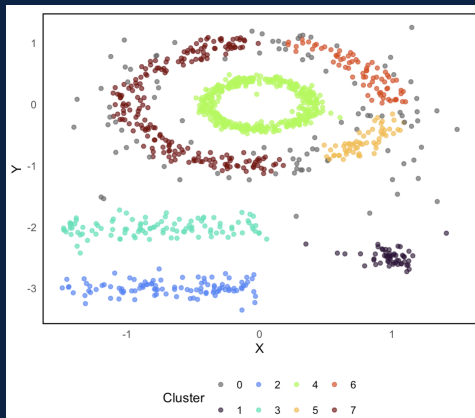
Advantages

- Only requires `minPts` (no ϵ needed)
- Produces cluster stability scores (higher = more stable)
- Provides probabilistic cluster assignments (like FCM from Workshop 3)
- Computes outlier scores for each point
- Handles clusters of varying densities

```
hdbscanResult <- hdbscan(shapesMatrix, minPts = 15)
max(hdbscanResult$cluster)
sum(hdbscanResult$cluster == 0)
head(hdbscanResult$membership_prob)
print(round(hdbscanResult$cluster_scores, 4))
shapesDf$hdbscanCluster <- factor(hdbscanResult$cluster)
shapesDf$hdbscanProb <- hdbscanResult$membership_prob
shapesDf$hdbscanOutlierScore <- hdbscanResult$outlier_scores
```

Some important outputs:

- `$cluster`: cluster labels (0 = noise)
- `$membership_prob`: soft membership probability per point
- `$cluster_scores`: stability of each cluster
- `$outlier_scores`: how likely an outlier each point is (more or less)

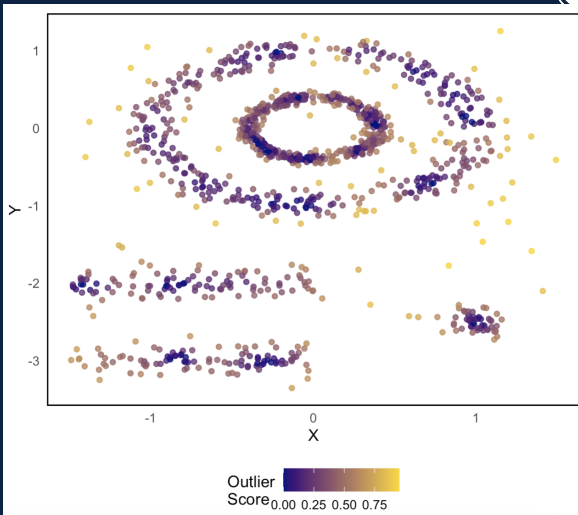


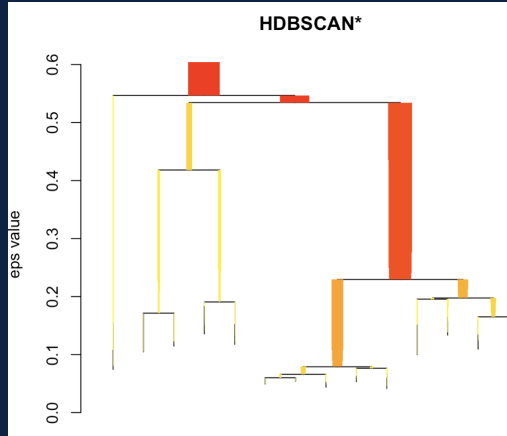
HDBSCAN Outlier Scores



```
ggplot(shapesDf, aes(x = x, y = y,  
  color = hdbscanOutlierScore)) +  
  geom_point(size = 1.5, alpha = 0.8) +  
  scale_color_gradient(low = "navy",  
    high = "gold",  
    name = "Outlier\nScore") +  
  labs(title = "HDBSCAN Outlier Scores",  
    x = "X", y = "Y")
```

Higher outlier scores indicate points that are more isolated from dense regions. This provides a continuous measure rather than a binary noise/cluster label.





The condensed tree shows how clusters form and split at different density levels. Think back to dendrograms from Workshop 1!

Clustering Performance Comparison



```
trueLabels <- as.numeric(shapesDf$trueCluster)
ariKmeans <- computeARI(trueLabels, as.numeric(shapesDf$kmeansCluster))
nmiKmeans <- computeNMI(trueLabels, as.numeric(shapesDf$kmeansCluster))
ariDbscan <- computeARI(trueLabels, as.numeric(shapesDf$dbscanCluster))
nmiDbscan <- computeNMI(trueLabels, as.numeric(shapesDf$dbscanCluster))
ariHdbscan <- computeARI(trueLabels, as.numeric(shapesDf$hdbscanCluster))
nmiHdbscan <- computeNMI(trueLabels, as.numeric(shapesDf$hdbscanCluster))
```

Density-based methods drastically outperform K -means on non-convex cluster shapes.

	ARI	NMI
K -Means	0.216	0.436
DBSCAN	0.963	0.934
HDBSCAN	0.732	0.796

	DBSCAN	OPTICS	HDBSCAN
Parameters	ϵ, minPts	$\epsilon_{\text{max}}, \text{minPts}$	minPts
Varying density	No	Yes	Yes
Hierarchy	No	Reachability plot	Condensed tree
Soft assignments	No	No	Yes
Outlier scores	Binary	No	Continuous
Speed	Fast	Moderate	Moderate

Real GPS Data from Chicago Crime Reports

Task is to identify spatial clusters (hotspots) of reported crimes using GPS coordinates.

Why density-based?

- Crime clusters have arbitrary shapes (not spherical)
- Isolated incidents are natural (outliers)
- No prior knowledge of number of hotspots
- Dense urban cores compared to sparse suburban areas

Data is from the city of Chicago open data portal (data.cityofchicago.org)

Preparing Crime Data



```
library(data.table)

url <- paste0(
  "https://data.cityofchicago.org/",
  "resource/ijzp-q8t2.csv?$limit=5000")
crimeRaw <- fread(url)

gps <- crimeRaw[, .(
  lat = as.numeric(latitude),
  lon = as.numeric(longitude),
  primaryType = primary_type,
  description = description,
  locationDesc = location_description,
  arrest = arrest,
  date = date)]

gps <- gps[!is.na(lat) & !is.na(lon)]

table(gps$primaryType)
```

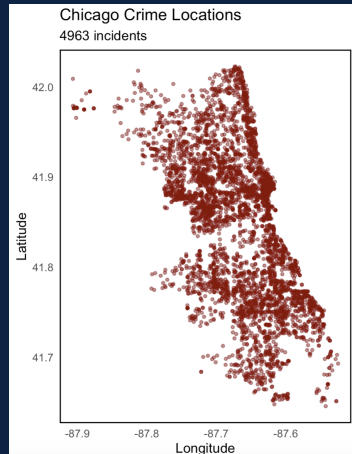
ARSON	ASSAULT
6	439
BATTERY	BURGLARY
883	227
CONCEALED CARRY LICENSE VIOLATION	CRIMINAL DAMAGE
3	514
CRIMINAL SEXUAL ASSAULT	CRIMINAL TRESPASS
33	137
DECEPTIVE PRACTICE	HOMICIDE
258	12
INTERFERENCE WITH PUBLIC OFFICER	INTIMIDATION
30	5
LIQUOR LAW VIOLATION	MOTOR VEHICLE THEFT
5	438
NARCOTICS	OFFENSE INVOLVING CHILDREN
184	40
OTHER OFFENSE	PROSTITUTION
388	4
PUBLIC PEACE VIOLATION	ROBBERY
19	103
SEX OFFENSE	STALKING
35	11
THEFT	WEAPONS VIOLATION
1128	98

Raw Crime Locations



```
ggplot(gps, aes(x = lon, y = lat)) +  
  geom_point(alpha = 0.5, size = 1,  
            color = "darkred") +  
  coord_fixed(ratio = 1.3) +  
  labs(title = "Chicago Crime Locations",  
       subtitle = paste(nrow(gps), "incidents"),  
       x = "Longitude", y = "Latitude")
```

Clear density variation: dense urban core with sparse suburban pockets. Perfect for density-based clustering.



Scaling GPS Coordinates to Metres

GPS coordinates are on a spherical Earth, so we approximate distances in metres:

- 1 degree latitude ≈ 111 km (everywhere)
- 1 degree longitude $\approx 111 \cos(\phi)$ km at latitude ϕ
- At Chicago ($\phi \approx 41.8^\circ$): 1 degree longitude ≈ 82.5 km

This gives us ε in interpretable units (metres).

```
latScale <- 111000
lonScale <- 111000 * cos(41.8 * pi / 180)
gps$latScaled <- (gps$lat - min(gps$lat)) * latScale
gps$lonScaled <- (gps$lon - min(gps$lon)) * lonScale
gpsMatrix <- as.matrix(gps[, .(lonScaled, latScaled)])
```

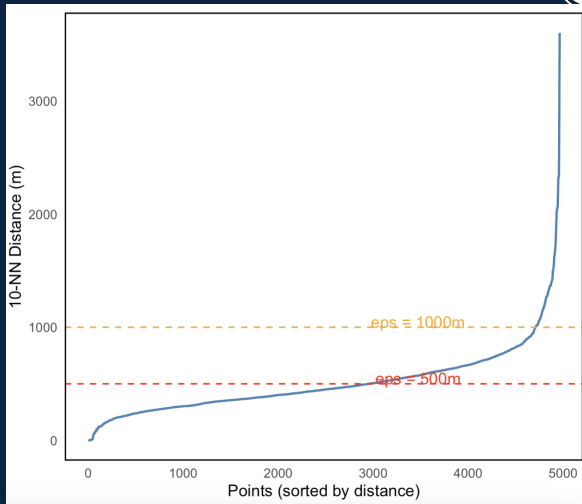
k -Distance Plot for Crime Data



```
k <- 10
kdist <- kNNdist(gpsMatrix, k = k)
kdistSorted <- sort(kdist)

kdistDf <- data.frame(
  index = 1:length(kdistSorted),
  distance = kdistSorted)

ggplot(kdistDf, aes(x = index, y = distance)) +
  geom_line(color = "steelblue",
            linewidth = 0.8) +
  geom_hline(yintercept = 300,
             linetype = "dashed",
             color = "red") +
  geom_hline(yintercept = 500,
             linetype = "dashed",
             color = "orange") +
  labs(x = "Points (sorted by distance)",
       y = paste0(k, "-NN Distance (m)"))
```



DBSCAN Crime Hotspots



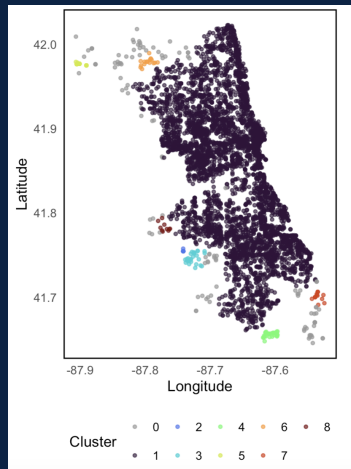
```
epsMeters <- 1000
minPts <- 10

dbscanResult <- dbscan::dbscan(gpsMatrix,
  eps = epsMeters, minPts = minPts)

max(dbscanResult$cluster)
sum(dbscanResult$cluster == 0)
print(table(dbscanResult$cluster))

gps$dbscanCluster <- factor(dbscanResult$cluster)

ggplot(gps, aes(x = lon, y = lat,
  color = dbscanCluster)) +
  geom_point(alpha = 0.7, size = 1) +
  coord_fixed(ratio = 1.3)
```

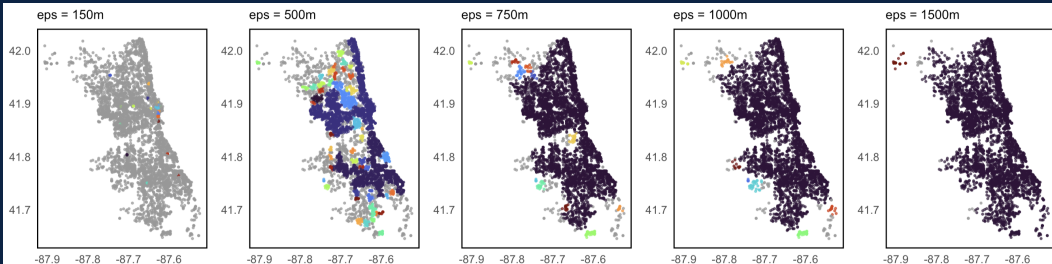


The core urban area forms a large cluster, with smaller pockets in suburban commercial centres. Grey points are isolated incidents classified as noise.

Effect of ϵ on Crime Hotspots



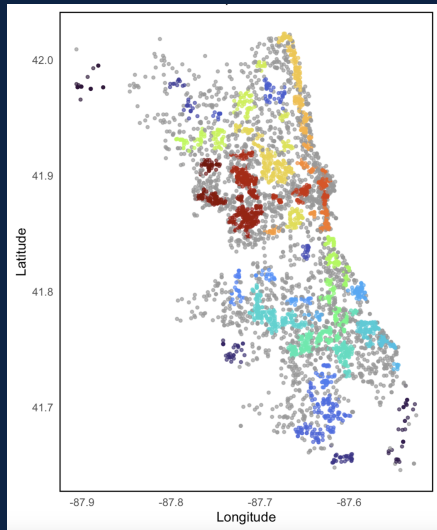
```
epsValues <- c(150, 500, 750, 1000, 1500); epsPlots <- list()
for (i in seq_along(epsValues)) {
  eps <- epsValues[i]
  dbTemp <- dbscan::dbscan(gpsMatrix, eps = eps, minPts = minPts)
  gps$tempCluster <- factor(dbTemp$cluster)
  nCl <- max(dbTemp$cluster)
  nNoise <- sum(dbTemp$cluster == 0)
  epsPlots[[i]] <- ggplot(gps, aes(x = lon, y = lat, color = tempCluster)) +
    geom_point(alpha = 0.7, size = 0.5) +
    labs(title = paste0("eps = ", eps, "m"))
}
grid.arrange(grobs = epsPlots, ncol = 2)
```



HDBSCAN Crime Hotspots



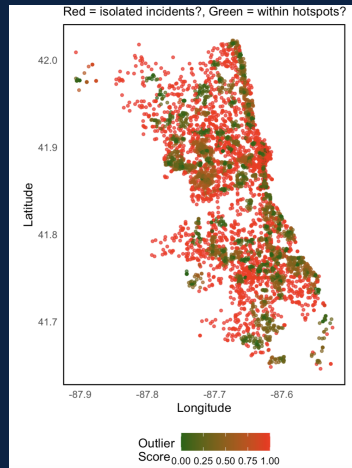
```
hdbscanResult <- hdbscan(gpsMatrix,  
                          minPts = minPts)  
  
max(hdbscanResult$cluster)  
sum(hdbscanResult$cluster == 0)  
print(table(hdbscanResult$cluster))  
  
gps$hdbscanCluster <-  
  factor(hdbscanResult$cluster)  
gps$outlierScore <-  
  hdbscanResult$outlier_scores  
gps$membershipProb <-  
  hdbscanResult$membership_prob
```



HDBSCAN Outlier Scores on Crime Data

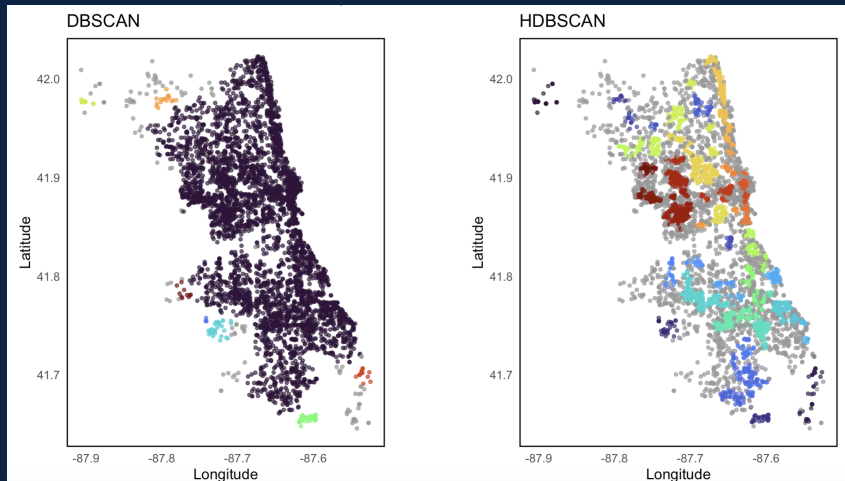


```
ggplot(gps, aes(x = lon, y = lat,  
               color = outlierScore)) +  
  geom_point(alpha = 0.7, size = 1) +  
  scale_color_gradient(low = "darkgreen",  
                      high = "red",  
                      name = "Outlier\nScore") +  
  coord_fixed(ratio = 1.3) +  
  labs(title = "HDBSCAN Outlier Scores",  
       subtitle = paste("Red = isolated?",  
                        "Green = within hotspots?"),  
       x = "Longitude", y = "Latitude")
```



Red = isolated incidents. Green = within dense hotspots. This provides actionable information for resource allocation.

Final Comparison



Where's the Performance Metrics?



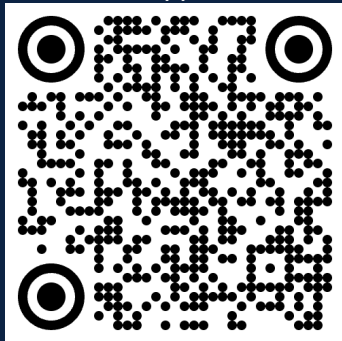
- Earlier workshops introduced many clustering and classification performance metrics.
- For this application, we did not use true class labels, leaving interpretation to visualization.
- For example, DBSCAN on Chicago crime data produces one large dense central cluster and smaller clusters on the outskirts—do these correspond to suburban commercial or industrial areas? What is located there?
- In contrast, HDBSCAN finds over 80 clusters and treats the space between them as outliers. Do these dense regions share features (e.g., proximity to metro stops), or do we need to examine additional data?
- You can also apply any of the previously defined clustering performance metrics to analyze these results.

- Density-based clustering finds clusters as dense regions separated by sparse regions
- DBSCAN uses ϵ and `minPts` to classify core, border, and noise points
- No need to specify K : number of clusters emerges from the data
- k -distance plot helps select ϵ (look for the elbow, like WCSS)
- OPTICS creates a reachability ordering, allowing cluster extraction at multiple scales
- HDBSCAN requires only `minPts` and provides stability scores, soft memberships, and outlier scores
- Outlier detection is built-in, unlike K -means, FCM, or GMM

Next Workshop: Feb 23, 3:30 PM

Additional Questions?

Book an Appointment!



Graph-Based Clustering

- > Workshop 2 (K -means) had circular cluster and hard partition limitations.
- > Workshop 3 (FCM) addressed the hard partition limitation.
- > Workshop 4 (GMM) addressed the circular cluster limitation.
- > Workshop 5 (DBSCAN) addressed shape constraints in general
- > Next workshop will switch gears and use graph connectivity and spectral methods to define clusters.

Thank You!

Questions?

Workshop Materials:

<https://csc-ubc-okanagan.github.io/workshops/>

Contact:

jesse.ghashti@ubc.ca